

TP : Génération de positions d'échecs (mats et puzzles) avec des GAN

Objectif :

Utiliser un modèle GAN pour générer automatiquement des positions plausibles d'échecs qui représentent :

- Des mats en 1 coup
 - Des puzzles tactiques simples (clouage, fourchette, enfilade, etc.)
-

1. Données

a. Source :

- [Lichess](#) : base de données de puzzles (disponible en CSV/PGN via Lichess database)
- Utiliser des positions FEN (Forsyth-Edwards Notation)

b. Prétraitement :

- Filtrer les puzzles : garder les puzzles de mat en 1 coup par exemple
 - Représenter chaque position FEN sous forme d'un **tensor/image** 8x8xN (canaux pour type de pièce, couleur...)
-

2. Architecture GAN

a. Générateur :

- Prend un vecteur latent z (bruit aléatoire) $\rightarrow$ génère une matrice représentant une position (FEN encodée)

b. Discriminateur :

- Prend une position (vraie ou générée) et prédit si elle est "réelle" (issue de puzzles existants) ou "fausse" (générée)

c. Pertes :

- Fonction perte classique GAN (`BinaryCrossEntropy`)
 - Option : introduire une contrainte pour forcer à générer des positions valides (via règles légales d'échecs)
-

3. Évaluation

a. Validité :

- Vérifier si la position est légale (via `chess.Board().is_valid()`)

b. Jouabilité :

- Vérifier s'il existe un mat en 1 via un moteur (comme `stockfish`) ou `python-chess`

c. Visualisation :

- Afficher les positions générées sur un échiquier
-

4. Outils utilisés

- `python-chess` pour manipulation des échiquiers
 - `tensorflow` pour implémenter le GAN
 - `stockfish` pour valider les mats générés
 - Option : notebook Jupyter pour l'interface
-

5. Extensions possibles

- Générer des puzzles de mat en 2 coups
- Générer des positions avec un thème tactique spécifique (fourchette, sacrifice, etc.)
- Ajouter un classificateur pour typer le puzzle généré (par thème)